

RETRIEVAL-AUGMENTED GENERATION

RAG na prática: o Agente RAG Univille

Arquitetura, pipeline e decisões técnicas de um assistente acadêmico construído com Python, LangChain, ChromaDB e React.

github.com/Plusnar/Agente-RAG-Univille

assistente-univille.vercel.app (ao vivo)

Gerar respostas ancoradas em documentos reais

RAG combina busca por similaridade semântica com geração de texto por LLM. Em vez de depender só do conhecimento pré-treinado do modelo, o sistema busca trechos relevantes em uma base própria e os injeta no prompt como contexto — reduzindo alucinações e permitindo respostas verificáveis e atualizáveis.



1. Indexação

Documentos são divididos em chunks e convertidos em vetores (embeddings).



2. Recuperação

A pergunta do usuário é vetorizada e comparada aos chunks mais próximos.



3. Geração

O LLM responde usando apenas os chunks recuperados como contexto.

Assistente Univille: contexto e objetivo

Assistente conversacional que responde perguntas de alunos com base em documentos oficiais da universidade — editais, matrizes curriculares e regulamentos — enviados pela própria instituição.



Fonte de verdade

Editais, matrizes curriculares e regulamentos enviados via painel admin.



Fidelidade ao contexto

Personalidade simpática e universitária, sem nunca abandonar a fidelidade ao documento-fonte.



Memória curta

Mantém contexto de continuidade entre perguntas em uma mesma conversa.



Gestão de conteúdo

Admin pode enviar, indexar e remover documentos pela própria interface.

As peças por trás do agente



FastAPI

Backend Python servindo API e o frontend de produção.



LangChain

Orquestra o pipeline de busca e geração de respostas.



ChromaDB

Banco vetorial local e persistente para os embeddings.



Cohere

embed-multilingual-v3.0 para embeddings e geração padrão.



OpenAI (opcional)

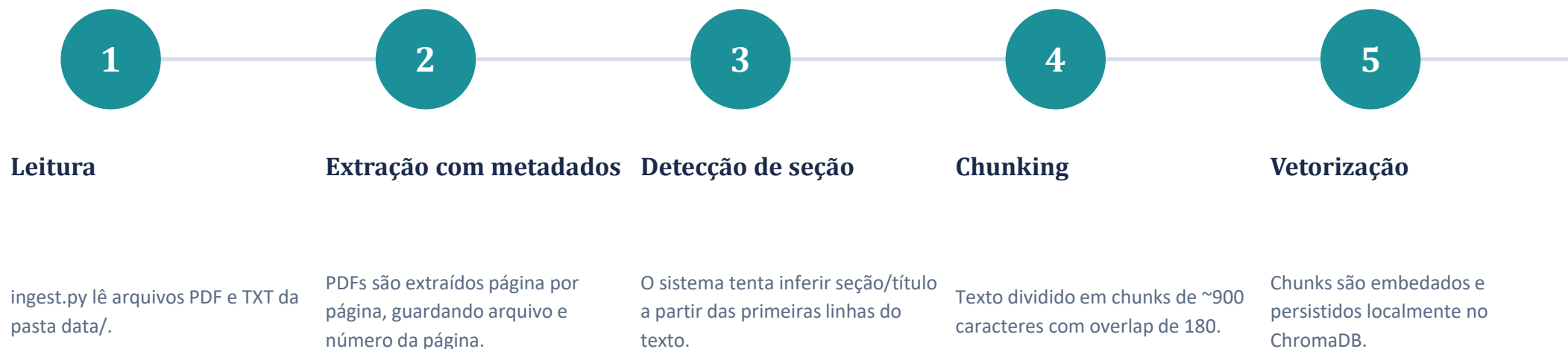
Provedor alternativo configurável para embeddings.



React + Vite

Interface do usuário, compilada e servida pelo FastAPI.

De PDF/TXT a vetores no ChromaDB



Ponto de atenção identificado no projeto: o tamanho de chunk (~900 chars) pode estourar limites de token dependendo do modelo — ajuste em avaliação.

Como uma pergunta vira resposta



Características-chave do retrieval

- Top-k = 5 chunks mais relevantes por pergunta (rag.py).
- Geração restrita ao contexto recuperado — reduz alucinação fora do escopo institucional.
- Embeddings multilíngues (Cohere embed-multilingual-v3.0), adequados ao português.
- Memória curta entre turnos para lidar com perguntas de continuidade.
- Persona 'simpática e universitária' aplicada na camada de prompt, sem sobrepor a fidelidade ao contexto.

FastAPI + React em um único deploy

O FastAPI serve tanto a API do agente quanto o build de produção do frontend React, simplificando o deploy em um único processo (ex.: uvicorn app:app).



Painel admin e ciclo de vida dos documentos



Acesso restrito

- Credenciais via variáveis de ambiente: ADMIN_USERNAME e ADMIN_PASSWORD.
- Chaves de API (Cohere/OpenAI) nunca versionadas — modelo em .env.example.
- Apenas o admin envia, indexa ou remove documentos da base.
- Remoção de documento apaga o arquivo e reindexa automaticamente os restantes.

Ciclo de vida de um documento

- 1 Upload de PDF/TXT pela interface admin
- 2 Clique em "Salvar documentos" → arquivo vai para data/
- 3 Clique em "Indexar documentos" → roda o pipeline de chunking + embeddings
- 4 Documento disponível para consulta pelos alunos
- 5 Remoção seleciona arquivos → exclui de data/ e reindexa o restante automaticamente

Desafios encontrados e próximos passos



Tamanho de chunk

Chunks de ~900 caracteres com overlap de 180 podem estourar limites de token de certos modelos — calibração pendente.



Em produção na Vercel

App já hospedado em `assistente-univille.vercel.app`, exigindo adaptar a persistência do ChromaDB ao ambiente serverless.



Multi-provedor de IA

Suporte a Cohere e OpenAI via variáveis de ambiente, permitindo trocar embeddings sem alterar código.

Roadmap técnico

- Recalibrar tamanho/overlap de chunk por modelo de embedding
- Garantir persistência do ChromaDB no ambiente serverless da Vercel
- Expandir cobertura de testes do pipeline de ingestão
- Métricas de qualidade de resposta (relevância dos chunks recuperados)
- Ampliar memória de conversa além do contexto curto atual

Resumo

Um RAG enxuto, focado e contextual

- Pipeline completo: ingestão → chunking → embeddings → retrieval → geração contextual
- Stack acessível e econômica: Python, FastAPI, ChromaDB local, Cohere/OpenAI, React
- Governança simples de conteúdo via painel admin, com reindexação automática

github.com/Plusnar/Agente-RAG-Univille

assistente-univille.vercel.app